CLOUD SOLUTION ARCHITECT · PORTFOLIO PROJECT

# Helios

End-to-End Architecture of the Cloud-Native LLM Orchestration Platform — how a request flows from edge to model to storage, and why every component is there.

| | | |
|---|---|---|
| **9**<br>request-path layers | **3**<br>cross-cutting planes | **56**<br>components explained |
| **40+**<br>tools & technologies | **14**<br>steps, edge → model | **5**<br>polyglot datastores |

## 01 The request, end to end

Follow one job through the platform. A client submits it at the edge; the gateway accepts it instantly and returns an id; it is queued on the event backbone; a worker runs the agent loop against the model with resilience wrapped around every call; results stream back and persist across the polyglot stores. Each step below is one component in the path.

1. **Cloudflare CDN** — Serves cached content close to users and absorbs traffic before it hits our servers.
2. **AWS WAF** — Blocks malicious requests (SQL injection, bad bots) before they reach the app.
3. **Ingress controller** — Routes each incoming request to the correct internal service by host/path, terminating TLS.
4. **Next.js · React · TypeScript** — Where users submit jobs, watch progress live, and view queue / health / usage.
5. **FastAPI gateway** — Accepts a job and instantly returns 202 + a job id, so the user never waits.
6. **Keycloak · OAuth2/OIDC/JWT** — Authenticates every caller and issues tokens that say what they're allowed to do.
7. **Provider registry & selector** — Picks a healthy provider for each job (round-robin / weighted / least-loaded).
8. **Rate limiter** — Keeps our call rate within each provider's limits so we're never throttled or banned.
9. **Circuit breaker** — Stops hammering a provider that's already failing, then probes to see if it recovered.
10. **Kafka / Redpanda** — The queue that decouples the fast gateway from the slower workers; jobs wait here safely.
11. **Go workers** — Pulls jobs off Kafka and runs the agent to completion, many at once.
12. **Agent loop** — The core routine that lets the model break a goal into steps and solve it.
13. **Anthropic Claude API** — Does the actual reasoning and tool-use for every job (Messages API).
14. **PostgreSQL** — The source of truth for jobs, tenants, quotas and the billing ledger.

## 02 Architecture map

The full system as stacked layers — the request flows top to bottom; the three cross-cutting planes (platform, delivery, observability) support every layer above. Logos identify each product; symbols mark patterns.

**LAYER 1 Client & Edge** — ingress · TLS · protection

- Cloudflare CDN
- AWS WAF
- Ingress controller
- Route 53 · DNS
- cert-manager / ACM

▼

**LAYER 2 Presentation** — dashboard · live view

- Next.js · React · TypeScript
- SSE live stream

▼

**LAYER 3 API Gateway** — accept · authenticate · meter

- FastAPI gateway
- Idempotency + OCC
- Keycloak · OAuth2/OIDC/JWT
- Quotas + usage ledger

▼

**LAYER 4 Control plane** — routing · experiments · provisioning

- Provider registry & selector
- Experimentation / feature flags
- Preview-env provisioner (Operator)

▼

**LAYER 5 Resilience layer** — wraps every provider call

- Rate limiter
- Circuit breaker
- Retry + backoff + jitter
- Fallback chain
- Load shedding

▼

**LAYER 6 Event backbone** — durable · decoupled · replayable

- Kafka / Redpanda
- Schema Registry
- Outbox pattern
- Dead-letter queue
- Debezium · CDC

▼

**LAYER 7 Worker fleet · Go** — the agent runtime

- Go workers
- Agent loop
- Tool / function calling
- Token budget
- RAG retrieval
- gRPC + Protobuf
- Istio service mesh

▼

**LAYER 8 LLM providers** — the intelligence

- Anthropic Claude API

▼

LAYER 9 **Storage plane · polyglot** — right store for each data shape

PostgreSQL · MongoDB · Redis · pgvector / Qdrant · S3 / MinIO · ClickHouse

▽ SPANS EVERY LAYER ABOVE ▽

CROSS-CUTTING **Platform & runtime** — where everything runs

Kubernetes · EKS + GKE · Helm + Kustomize · Custom Operator + CRD · KEDA · HashiCorp Vault

CROSS-CUTTING **Delivery & supply chain** — how code reaches production

GitHub Actions + OIDC · Argo CD · GitOps · Argo Rollouts · Terraform / Crossplane · SBOM · cosign · SLSA

CROSS-CUTTING **Observability & SRE** — is it healthy? is it fast?

OpenTelemetry · Prometheus · Loki · Grafana · Pyroscope · SLOs + burn-rate alerts · Chaos Mesh · k6

# 03 Component reference

Every component in plain English — **what it is**, **its role** in the layer, **why we chose it**, and its trade-offs. Written so a non-technical reader can follow, and you can defend each choice.

## Client & Edge

### Cloudflare CDN

CLIENT & EDGE

**What:** A global content-delivery & edge network with 300+ locations worldwide.

**Role:** Serves cached content close to users and absorbs traffic before it hits our servers.

> **Why chosen:** Massive free tier, built-in DDoS protection and WAF, and very low latency.

+ Global edge = fast for everyone
+ Built-in DDoS + bot protection
+ Cheap / generous free tier

– One more vendor to manage
– Cache invalidation needs care

### AWS WAF

CLIENT & EDGE

**What:** A Web Application Firewall that inspects incoming HTTP requests.

**Role:** Blocks malicious requests (SQL injection, bad bots) before they reach the app.

> **Why chosen:** Fully managed, ships with common attack rule-sets, integrates with our AWS load balancer.

+ Managed OWASP rule-sets
+ Blocks attacks at the edge

– Per-rule cost
– Needs tuning to avoid false blocks

### Ingress controller

CLIENT & EDGE

**What:** A reverse proxy that is the single front door into the Kubernetes cluster.

**Role:** Routes each incoming request to the correct internal service by host/path, terminating TLS.

> **Why chosen:** Nginx ingress is mature, battle-tested and flexible.

+ Mature & flexible routing
+ Central place for TLS & rules

– A choke point to scale & secure carefully

### Route 53 · DNS

CLIENT & EDGE

**What:** Amazon's managed DNS service.

**Role:** Turns our domain name into the IP address of the load balancer, with health-based failover.

> **Why chosen:** AWS-native, supports latency/geo routing and health checks for multi-region.

+ Health-checked failover
+ Latency & geo routing

– Ties DNS to AWS

### cert-manager / ACM

CLIENT & EDGE

**What:** Automated TLS-certificate issuance and renewal.

**Role:** Gives every endpoint HTTPS and auto-renews certs so they never expire.

> **Why chosen:** Automates free ACME/Let's Encrypt certs inside the cluster — no manual renewals.

+ Auto-renew = no expiry outages
+ Free certificates

– Depends on the cluster & DNS setup

## Presentation

### Next.js · React · TypeScript

PRESENTATION

**What:** The web dashboard, built with React + Next.js and typed with TypeScript.

**Role:** Where users submit jobs, watch progress live, and view queue / health / usage.

> **Why chosen:** Server-side rendering, great developer experience, and types that catch bugs early.

+ Fast, SEO-friendly rendering
+ Types prevent whole classes of bugs
+ Huge ecosystem

– Build tooling is complex
– Frontend churn

### SSE live stream

PRESENTATION

**What:** Server-Sent Events — a one-way live stream over plain HTTP.

**Role:** Pushes the agent's thinking to the browser token-by-token as it happens.

> **Why chosen:** Simpler than WebSockets when you only need server→browser streaming.

+ Simple, HTTP-native, auto-reconnect

– One-way only
– Browser connection limits

## API Gateway

### FastAPI gateway
API GATEWAY

**What:** An async Python web framework serving as the platform's front door.

**Role:** Accepts a job and instantly returns 202 + a job id, so the user never waits.

**Why chosen:** Async-native (handles many requests at once), typed via Pydantic, auto-generates API docs.

+ Very fast & async
+ Typed request validation
+ Auto OpenAPI docs

– Requires async discipline
– Younger than Django/Flask

### Idempotency + OCC
API GATEWAY

**What:** Idempotency keys plus optimistic concurrency control.

**Role:** Makes retries safe (no double-processing) and prevents two writers from clobbering data.

**Why chosen:** Clients and networks retry automatically — the system must never act twice on one request.

+ Safe retries
+ No lost updates

– Extra key storage & careful design

### Keycloak · OAuth2/OIDC/JWT
API GATEWAY

**What:** An open-source identity provider speaking OAuth2 / OIDC / JWT.

**Role:** Authenticates every caller and issues tokens that say what they're allowed to do.

**Why chosen:** Full-featured, standards-based, self-hosted — no per-user SaaS fees.

+ Industry-standard auth
+ No per-user cost
+ Self-hosted control

– Heavier to operate yourself

### Quotas + usage ledger
API GATEWAY

**What:** Per-tenant rate/usage limits plus an append-only usage record.

**Role:** Stops one customer starving the others and records usage for billing.

**Why chosen:** Multi-tenant fairness and accurate billing both need it.

+ Fair sharing
+ Billing-grade usage data

– Accounting adds complexity

## Control plane

### Provider registry & selector
CONTROL PLANE

**What:** A live registry of available LLM providers plus a routing strategy.

**Role:** Picks a healthy provider for each job (round-robin / weighted / least-loaded).

**Why chosen:** We need to route, weight and fail over between providers at runtime, not in code.

+ Flexible routing
+ Easy failover

– Must track provider health accurately

### Experimentation / feature flags
CONTROL PLANE

**What:** An A/B-testing framework with feature flags.

**Role:** Rolls a new agent config or model out to a small % of traffic and measures results.

**Why chosen:** Lets us change the product with data and roll back instantly if a metric drops.

+ Safe, measurable rollouts
+ Instant rollback

– Flag sprawl
– Needs statistical rigor

### Preview-env provisioner (Operator)
CONTROL PLANE

**What:** A custom Kubernetes Operator + CRD we write ourselves.

**Role:** Spins up and tears down isolated per-tenant environments on demand.

**Why chosen:** Automates what would otherwise be manual, error-prone ops work.

+ Self-service environments
+ Consistent & repeatable

– Real engineering effort to build

## Resilience layer

### Rate limiter
RESILIENCE LAYER

**What:** A token-bucket limiter backed by Redis + Lua.

**Role:** Keeps our call rate within each provider's limits so we're never throttled or banned.

**Why chosen:** Providers enforce hard rate limits; we must self-regulate across many workers.

+ Protects providers
+ Fair distributed limiting
– Distributed counting is tricky

### Circuit breaker
RESILIENCE LAYER

**What:** A breaker with closed / open / half-open states.

**Role:** Stops hammering a provider that's already failing, then probes to see if it recovered.

**Why chosen:** Fail fast instead of piling requests onto a broken dependency.

+ Prevents cascading failures
+ Auto-recovery probing
– Thresholds need tuning

### Retry + backoff + jitter
RESILIENCE LAYER

**What:** Smart retries that wait longer each time, with randomness (jitter).

**Role:** Recovers from brief, transient errors without all workers retrying in sync.

**Why chosen:** Networks blip; naive retries cause a stampede — jitter spreads them out.

+ Higher success rate
+ Avoids thundering herd
– Can amplify load if misconfigured

### Fallback chain
RESILIENCE LAYER

**What:** An ordered list of backup providers.

**Role:** If the primary fails, quietly try the next one so the user still gets an answer.

**Why chosen:** Keeps the service useful during a partial outage.

+ Graceful degradation
– Can hide root causes — use carefully

### Load shedding
RESILIENCE LAYER

**What:** Deliberately dropping excess requests under overload.

**Role:** Protects the system so it stays up for most users instead of collapsing for all.

**Why chosen:** Partial service beats total meltdown when demand exceeds capacity.

+ System stays alive under spikes
– Some requests are rejected

## Event backbone

### Kafka / Redpanda
EVENT BACKBONE

**What:** A distributed, durable event log (Redpanda is a drop-in Kafka).

**Role:** The queue that decouples the fast gateway from the slower workers; jobs wait here safely.

**Why chosen:** Durable, ordered, replayable and high-throughput — the backbone of event-driven systems.

+ Durable & replayable
+ Scales to huge throughput
+ Decouples producers/consumers
– Operationally complex

### Schema Registry
EVENT BACKBONE

**What:** A registry of versioned event schemas.

**Role:** Lets producers and consumers evolve message formats without breaking each other.

**Why chosen:** In a big system, message shapes change — this makes that safe.

+ Safe schema evolution
+ Contract enforcement
– Extra infrastructure

### Outbox pattern
EVENT BACKBONE

**What:** A transactional outbox table + relay.

**Role:** Publishes an event in the same DB transaction as the data write — no lost or duplicate events.

**Why chosen:** Guarantees the database and the event stream never disagree.

+ Strong consistency
– Extra table + relay process

### Dead-letter queue
EVENT BACKBONE

**What:** A side queue for messages that repeatedly fail.

**Role:** Parks poison messages so one bad job doesn't block the whole stream.

**Why chosen:** Keeps the pipeline flowing while preserving failures for later inspection.

+ Isolates poison messages
– Needs monitoring & replay tooling

### Debezium · CDC

EVENT BACKBONE

**What:** Change-Data-Capture that tails the database's change log.

**Role:** Streams every DB change into the analytics warehouse in real time.

**Why chosen:** Gets analytics data without risky dual-writes from the app.

+ Real-time, decoupled analytics
– Connectors add ops overhead

## Worker fleet · Go

### Go workers

WORKER FLEET · GO

**What:** The worker service that does the heavy lifting, written in Go.

**Role:** Pulls jobs off Kafka and runs the agent to completion, many at once.

**Why chosen:** Go's goroutines make massive concurrency cheap; binaries are tiny and fast.

+ Cheap concurrency
+ Fast & low memory
+ Tiny container images
– A new language to learn

### Agent loop

WORKER FLEET · GO

**What:** The plan → act → observe reasoning cycle.

**Role:** The core routine that lets the model break a goal into steps and solve it.

**Why chosen:** This IS the product — an autonomous, tool-using agent.

+ Autonomous problem-solving
+ Uses tools & data
– Can loop or overspend if unbounded

### Tool / function calling

WORKER FLEET · GO

**What:** Structured 'function calling' the model can invoke.

**Role:** Lets the agent take real actions (search, fetch, compute), not just talk.

**Why chosen:** Bridges the model's reasoning to the outside world.

+ Turns text into real actions
– Inputs must be validated for safety

### Token budget

WORKER FLEET · GO

**What:** A cap on tokens (and thus cost/time) per job.

**Role:** Bounds how much a single job can spend before it must stop.

**Why chosen:** LLM calls cost real money — runaway agents must be contained.

+ Predictable cost
+ Prevents runaways
– May cut off long tasks

### RAG retrieval

WORKER FLEET · GO

**What:** Retrieval-Augmented Generation over your own data.

**Role:** Finds relevant documents by meaning and feeds them to the model for grounded answers.

**Why chosen:** Cuts hallucinations and keeps answers current without retraining.

+ Grounded, up-to-date answers
– Only as good as the retrieval

### gRPC + Protobuf

WORKER FLEET · GO

**What:** A fast, typed remote-procedure-call system.

**Role:** How internal services call each other efficiently with strict contracts.

**Why chosen:** Faster and stricter than REST for internal, high-volume service calls.

+ Fast binary protocol
+ Typed contracts
+ Streaming
– Not human-readable; extra tooling

### Istio service mesh

WORKER FLEET · GO

**What:** A service mesh that sits beside every service.

**Role:** Adds mutual-TLS encryption and traffic control between services without app code.

**Why chosen:** Security and traffic-shaping handled by the platform, not each app.

+ mTLS everywhere
+ Canary & retries at mesh level
+ Free observability
– Adds complexity & overhead

## LLM providers

### Anthropic Claude API
LLM PROVIDERS

**What:** The large language model that powers the agents.
**Role:** Does the actual reasoning and tool-use for every job (Messages API).

**Why chosen:** Strong tool-use, long context and reliable reasoning — ideal for agents.

+ Excellent tool-use & reasoning
+ Long context window
– External cost & latency
– Provider rate limits

## Storage plane · polyglot

### PostgreSQL
STORAGE PLANE · POLYGLOT

**What:** A rock-solid relational (SQL) database.
**Role:** The source of truth for jobs, tenants, quotas and the billing ledger.

**Why chosen:** ACID guarantees, decades of maturity, plus JSON and extensions when needed.

+ Reliable & consistent
+ Extremely versatile
– Write-scaling needs planning

### MongoDB
STORAGE PLANE · POLYGLOT

**What:** A document (NoSQL) database.
**Role:** Stores variable-shaped agent trajectories and payloads that don't fit tidy tables.

**Why chosen:** Flexible schema is perfect for deeply-nested, evolving agent data.

+ Flexible schema
+ Fast document reads
– Weaker multi-document transactions

### Redis
STORAGE PLANE · POLYGLOT

**What:** An in-memory data store.
**Role:** Cache, hot state, and the counters behind rate-limiting and circuit-breaking.

**Why chosen:** Microsecond latency for the things that must be instant.

+ Blazing fast
+ Very versatile
– Memory-bound
– Durability trade-offs

### pgvector / Qdrant
STORAGE PLANE · POLYGLOT

**What:** A vector database for embeddings.
**Role:** Stores the numeric 'meaning' of documents so RAG can find similar ones.

**Why chosen:** Enables semantic (meaning-based) search, not just keyword match.

+ Semantic similarity search
– Index tuning & memory cost

### S3 / MinIO
STORAGE PLANE · POLYGLOT

**What:** Object storage (MinIO is S3-compatible, self-hosted).
**Role:** Holds large artifacts and long-term archives cheaply.

**Why chosen:** Effectively infinite, durable and inexpensive for blobs.

+ Cheap, durable, scalable
– Higher latency than a DB

### ClickHouse
STORAGE PLANE · POLYGLOT

**What:** A columnar OLAP (analytics) database.
**Role:** The read-model / warehouse for fast analytics and dashboards.

**Why chosen:** Aggregates billions of rows in milliseconds — built for analytics.

+ Extremely fast analytics
– Not for transactional writes

## Platform & runtime

### Kubernetes · EKS + GKE
PLATFORM & RUNTIME

**What:** The container orchestrator that runs the whole fleet.
**Role:** Schedules, heals and scales every service across many machines.

**Why chosen:** The industry standard for running containers reliably at scale.

+ Self-healing & scaling
+ Portable across clouds
– Steep learning curve

### Helm + Kustomize
PLATFORM & RUNTIME

**What:** Packaging and configuration tools for Kubernetes.
**Role:** Bundle each app's manifests and tailor them per environment.

**Why chosen:** Standard way to template and reuse deployment config.

+ Reusable, templated deploys
– Templating can get complex

### Custom Operator + CRD
PLATFORM & RUNTIME

**What:** A controller we write that extends Kubernetes with our own resource type.

**Role:** Automates tenant creation and preview environments as native K8s objects.

**Why chosen:** Encodes our ops knowledge into the platform itself.

+ Automates complex ops
+ Top-tier skill to show
– Non-trivial to build correctly

### KEDA
PLATFORM & RUNTIME

**What:** Event-driven autoscaler for Kubernetes.

**Role:** Adds or removes workers based on how many jobs are waiting in Kafka.

**Why chosen:** Scales on real backlog, not just CPU.

+ Scale on queue depth
+ Scale to zero
– Another component to run

### HashiCorp Vault
PLATFORM & RUNTIME

**What:** A secrets manager and PKI.

**Role:** Issues short-lived database credentials and certificates on demand.

**Why chosen:** Dynamic, audited secrets beat long-lived passwords in config.

+ Dynamic, short-lived secrets
+ Full audit trail
– Operationally involved

## Delivery & supply chain

### GitHub Actions + OIDC
DELIVERY & SUPPLY CHAIN

**What:** The CI system that builds, tests and ships.

**Role:** Runs on every push and deploys to the cloud using short-lived OIDC tokens (no stored keys).

**Why chosen:** Keyless cloud auth removes the biggest secret-leak risk in CI.

+ Keyless, safer deploys
+ Tight GitHub integration
– YAML sprawl at scale

### Argo CD · GitOps
DELIVERY & SUPPLY CHAIN

**What:** A GitOps continuous-delivery tool.

**Role:** Keeps the cluster exactly matching what's declared in Git, automatically.

**Why chosen:** Git becomes the single source of truth; drift self-corrects.

+ Declarative & auditable
+ Auto drift-correction
– Everything must live in Git

### Argo Rollouts
DELIVERY & SUPPLY CHAIN

**What:** Progressive-delivery controller.

**Role:** Releases new versions gradually (canary / blue-green) and auto-rolls-back on bad metrics.

**Why chosen:** De-risks every deploy by testing on a slice of real traffic first.

+ Safe, gradual releases
+ Metric-based auto-rollback
– Adds release-config complexity

### Terraform / Crossplane
DELIVERY & SUPPLY CHAIN

**What:** Infrastructure-as-Code tools.

**Role:** Define all cloud infrastructure (network, clusters, databases) as versioned code.

**Why chosen:** Reproducible, reviewable infrastructure instead of manual clicking.

+ Reproducible infra
+ Peer-reviewable changes
– State management care needed

### SBOM · cosign · SLSA
DELIVERY & SUPPLY CHAIN

**What:** Software supply-chain security tooling.

**Role:** Lists everything in each image, signs it, and proves how it was built.

**Why chosen:** Lets us verify nothing was tampered with before it runs in production.

+ Tamper-evident builds
+ Provenance & trust
– Extra CI steps to maintain

## Observability & SRE

### OpenTelemetry
OBSERVABILITY & SRE

**What:** A vendor-neutral standard for traces, metrics and logs.

**Role:** Tags each request so we can follow it across every service (async, Kafka, gRPC).

**Why chosen:** One standard instrumentation instead of per-vendor agents.

+ Vendor-neutral
+ End-to-end request traces
– Instrumentation effort

### Prometheus
OBSERVABILITY & SRE

**What:** A metrics database and alerting engine.

**Role:** Collects numbers (latency, error rate, queue depth) and fires alerts on rules.

**Why chosen:** The de-facto standard for cloud-native metrics.

+ Powerful queries
+ Strong alerting
– Long-term storage needs add-ons

### Loki
OBSERVABILITY & SRE

**What:** A log aggregation system.

**Role:** Centralizes logs from every service so we can search them in one place.

**Why chosen:** Cheap, label-based logs that pair naturally with Grafana.

+ Cheap log storage
+ Grafana-native
– Not full-text like Elasticsearch

### Grafana
OBSERVABILITY & SRE

**What:** The visualization layer.

**Role:** Dashboards that show the health of the entire system at a glance.

**Why chosen:** Single pane of glass over metrics, logs and traces.

+ One place for everything
+ Beautiful dashboards
– Dashboard maintenance

### Pyroscope
OBSERVABILITY & SRE

**What:** Continuous profiling.

**Role:** Shows exactly which code lines burn CPU/memory in production.

**Why chosen:** Finds performance hot-spots that metrics alone can't.

+ Pinpoints hot code paths
– Small runtime overhead

### SLOs + burn-rate alerts
OBSERVABILITY & SRE

**What:** Service Level Objectives with error budgets.

**Role:** Define 'good enough' reliability and alert only when we're burning the budget too fast.

**Why chosen:** Alerts on user-impact, not noise — the heart of SRE.

+ Meaningful, low-noise alerts
+ Aligns eng with users
– Requires honest target-setting

### Chaos Mesh
OBSERVABILITY & SRE

**What:** A chaos-engineering tool.

**Role:** Deliberately injects failures (kill pods, add latency) to prove resilience.

**Why chosen:** You only know it's resilient if you've tested failure on purpose.

+ Proves resilience
+ Finds weak spots early
– Must be run carefully

### k6
OBSERVABILITY & SRE

**What:** A load-testing tool.

**Role:** Simulates heavy traffic to find limits before real users do.

**Why chosen:** Scriptable, developer-friendly performance testing.

+ Realistic load tests
+ Scriptable in JS
– Test design takes effort